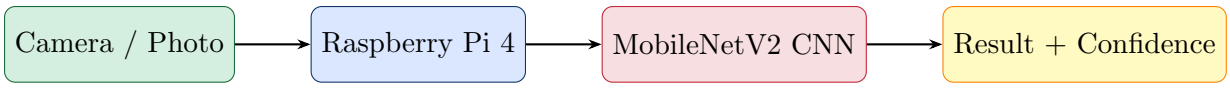
Pi Vision

CNN-Powered Object Recognition

on Raspberry Pi 4 Model B



Project:	Pi Vision – Edge AI Object Recognition
Hardware:	Raspberry Pi 4 Model B
Model:	MobileNetV2 (ONNX)
Interface:	Flask Web Application
Language:	Python 3.x
Classes:	1,000 ImageNet Categories

February 24, 2026

Contents

Chapter 1

Introduction

1.1 What is Pi Vision?

Pi Vision is a student-friendly artificial intelligence project that runs entirely on a Raspberry Pi 4 Model B. It uses a **Convolutional Neural Network (CNN)** called MobileNetV2 to identify objects in photographs. Students take a picture using their phone or laptop camera, upload it through a web browser, and within milliseconds the Pi returns a prediction of what object is in the image — along with a confidence percentage for the top five guesses.

The key feature that makes this project special is that *everything runs locally on the Pi.* No internet connection is required after the initial setup, no data is sent to any cloud server, and the entire AI model weighs only 14 MB. This is what engineers call **Edge AI** — running artificial intelligence directly on small, embedded devices rather than in the cloud.

1.2 Learning Objectives

By completing this project, students will understand:

- How Convolutional Neural Networks (CNNs) process and classify images
- What inference means in the context of machine learning
- How the Raspberry Pi works as a small Linux computer
- How to set up and SSH into a headless embedded device
- How to build and serve a Python web application using Flask
- The concept of Edge AI versus Cloud AI
- How ImageNet and transfer learning enable powerful models in small packages

1.3 System Overview

Full System Pipeline

User takes photo with phone → **Browser** uploads image to Pi over WiFi → **Flask server** receives image → **MobileNetV2 CNN** processes 224×224 pixels through 53 layers → **Softmax** converts outputs to probabilities → **Browser** displays top-5 predictions with confidence bars

Chapter 2

Raspberry Pi 4 — Hardware Guide

2.1 What is the Raspberry Pi 4?

The Raspberry Pi 4 Model B is a credit-card-sized single-board computer (SBC) developed by the Raspberry Pi Foundation in Cambridge, UK. Despite its small size, it is a fully functional Linux computer capable of running a complete desktop operating system, serving web applications, and even running neural network inference.

Component	Specification	Role in This Project
CPU	Broadcom BCM2711, 4-core ARM Cortex-A72 @ 1.8 GHz	Runs Python + CNN infe
RAM	2 GB / 4 GB / 8 GB LPDDR4 SDRAM	Holds model + web server
Storage	MicroSD card (16GB+)	Stores the OS, code, and r
WiFi	802.11ac dual-band	Connects to your home ne
USB	2× USB 3.0, 2× USB 2.0	For keyboard, mouse, USE
GPIO	40-pin header	For LEDs, sensors (future
Camera Port	CSI ribbon cable connector	For Pi Camera Module
Video Out	2× micro-HDMI	Connect to a monitor
Power	USB-C 5V / 3A	Must use official adapter
OS	Raspberry Pi OS (Debian Linux 64-bit)	Operating system

2.2 How the Pi Processes the CNN

The Raspberry Pi 4's ARM Cortex-A72 processor runs the MobileNetV2 inference using all four CPU cores. Each image inference takes approximately 150–400 milliseconds, which is fast enough for a responsive web application. The Pi does *not* have a dedicated GPU for this project (though one could add a Coral USB Accelerator to reduce inference time to under 10ms).

Power Supply is Critical

Always use the official Raspberry Pi USB-C 5V/3A power adapter. Using a phone charger or underpowered supply causes *undervoltage throttling* — the Pi slows its CPU to protect itself, making inference up to 4× slower. A yellow lightning bolt icon appears on screen if this happens.

2.3 Required Hardware Components

Item	Required?	Cost (approx.)	Notes
Raspberry Pi 4 Model B (4GB)	Yes	$55	2GB works but 4GB recom
MicroSD card 32GB (Class 10/A1)	Yes	$8	SanDisk or Samsung recom
Official USB-C 5V/3A power adapter	Yes	$8	Do not substitute
MicroSD card reader (for PC)	Yes	$6	To flash the OS
Ethernet cable	Recommended	$5	More reliable than WiFi fo
Pi Camera Module v2/v3	Optional	$25–35	Or use a USB webcam
USB webcam	Optional	$15+	Easier alternative to Pi Ca
Micro-HDMI to HDMI cable	Optional	$8	Only needed if using a mo

2.4 The GPIO Header

The 40-pin GPIO (General Purpose Input/Output) header along the edge of the Pi board allows connection to external electronics. While not used in the base Pi Vision project, students can extend the project by connecting:

- **LED strips** — light up different colors based on what object is detected
- **Buzzer** — make a sound when a specific object is found
- **OLED display** — show the prediction directly on a small screen
- **Servo motor** — physically point at detected objects

Chapter 3

Operating System and Configuration

3.1 Raspberry Pi OS

The Raspberry Pi runs **Raspberry Pi OS** (formerly called Raspbian), which is a customized version of Debian Linux designed specifically for the Pi's ARM processor. It includes Python 3, a desktop environment, and all the drivers needed for Pi peripherals like the camera module.

For this project, we use the **64-bit version** ("Bookworm") which is required for optimal performance with modern Python packages on the ARM Cortex-A72 processor.

3.2 Headless Setup (No Monitor Required)

A **headless** setup means running the Pi with no monitor, keyboard, or mouse. You control the Pi entirely from your PC using SSH (Secure Shell) over the network. This is the recommended approach for this project.

3.2.1 Step 1: Flashing the SD Card

1. Download **Raspberry Pi Imager** from `https://www.raspberrypi.com/software/`
2. Insert the MicroSD card into your PC's card reader
3. Open Imager, select *Raspberry Pi OS (64-bit)* as the OS
4. Click the **gear icon** (`Ctrl+Shift+X`) to open advanced settings
5. Configure the following *before* flashing:

Setting	Value
Hostname	`raspberrypi`
Enable SSH	Checked (use password authentication)
Username	`pi`
Password	(choose something you will remember)
Configure WiFi SSID	Your home WiFi network name
Configure WiFi Password	Your home WiFi password
WiFi Country	Your country code (e.g. US)

WiFi Name is Case-Sensitive

The WiFi SSID (network name) must match *exactly*, including capital letters and spaces. If your network is "MyHomeWiFi" you cannot type "myhomewifi".

3.2.2 Step 2: First Boot

Insert the flashed SD card into the Pi, connect power, and wait **90 seconds**. The Pi needs this time to resize the filesystem, generate SSH keys, and connect to WiFi. The green LED will blink during this process.

3.2.3 Step 3: SSH Connection

From your PC, open PowerShell (Windows) or Terminal (Mac/Linux):

```
ssh pi@raspberrypi.local
```

Listing 3.1: Connecting to the Pi via SSH

If this fails, find the Pi's IP address first:

```
# On Windows PowerShell
arp -a
# Look for a device with MAC prefix b8:27:eb or dc:a6:32

# Then connect using the IP directly
ssh pi@192.168.1.45
```

Listing 3.2: Finding the Pi's IP address

3.3 Understanding SSH

SSH stands for **Secure Shell**. It creates an encrypted tunnel between your PC and the Pi over the network. Once connected, every command you type in the terminal executes *on the Pi*, not on your local computer. This is how system administrators manage remote servers, and it is exactly what you are doing with the Pi.

Keep SSH Open During Development

Open two terminal windows — one for running the Flask app and one for editing files. This way you can restart the app without closing your editor session.

Chapter 4

Convolutional Neural Networks (CNNs)

4.1 What is a Neural Network?

A neural network is a mathematical model loosely inspired by the structure of the human brain. It consists of layers of **neurons** (mathematical functions) that transform input data into output predictions. The network *learns* by adjusting millions of internal numerical weights during a training process, where it sees millions of labelled examples.

4.2 What Makes a CNN Different?

A standard neural network treats every pixel in an image independently, which is extremely inefficient. A **Convolutional Neural Network** instead uses *convolutional filters* — small matrices that slide across the image and detect local patterns like edges, textures, and shapes. This gives CNNs two major advantages:

1. **Parameter efficiency:** A single filter learns a feature once and applies it everywhere in the image, massively reducing the number of weights needed.
2. **Spatial awareness:** CNNs understand that nearby pixels are related, which is fundamentally true for images.

4.3 How CNNs Build Understanding Layer by Layer

Shapes; [layer, fill=lightBlue!40, draw=darkBlue, right=0.6cm of l2] (l3) Layer 3
Object Parts; [layer, fill=piRed!15, draw=piRed, right=0.6cm of l3] (out) Output
1000 class
probabilities;

[arrow] (input) – (l1); [arrow] (l1) – (l2); [arrow] (l2) – (l3); [arrow] (l3) – (out);

Early layers detect simple features like horizontal or diagonal edges. Middle layers combine those edges into shapes like circles, rectangles, and curves. Deep layers combine shapes into recognizable object parts like “fur texture + pointy ear = cat”. The final layer maps these high-level features to one of the 1,000 output classes.

4.4 MobileNetV2 — The CNN Used in Pi Vision

MobileNetV2 is a CNN architecture developed by Google in 2018, specifically designed to run on mobile and embedded devices with limited computational resources. It achieves this through two key innovations:

4.4.1 Depthwise Separable Convolutions

Standard convolutions apply a single filter that processes all color channels simultaneously. MobileNetV2 splits this into two steps:

1. **Depthwise convolution:** Apply one filter per channel independently
2. **Pointwise convolution:** Combine channel outputs with a 1×1 filter

This reduces computation by a factor of approximately k^2 (where k is the filter size), typically achieving **8–9× speedup** with minimal accuracy loss.

4.4.2 Inverted Residuals with Linear Bottlenecks

MobileNetV2 introduces *inverted residual blocks* that first expand the number of channels, apply depthwise convolution in the high-dimensional space, then compress back down. This allows the network to extract rich features while keeping the *stored* representations compact.

4.4.3 MobileNetV2 Performance Specifications

Metric	Value
Top-1 Accuracy (ImageNet)	71.8%
Top-5 Accuracy (ImageNet)	91.0%
Number of Parameters	3.4 million
Model File Size	14 MB
Input Image Size	224×224 pixels
Number of Layers	53
Inference Time on Pi 4	∼150–400 ms
Inference Time on PC (i5/i7)	∼20–50 ms

4.5 ImageNet — The Training Dataset

MobileNetV2 was trained on **ImageNet**, one of the most famous datasets in computer vision. It contains:

- **1.2 million** training images
- **50,000** validation images
- **1,000** object categories (classes)
- Images collected from across the internet and hand-labelled

Training MobileNetV2 on ImageNet from scratch would take weeks on a cluster of high-end GPUs. By downloading a *pre-trained* model, we get all that knowledge in a 14 MB file for free — this concept is called **transfer learning**.

4.6 The 1,000 Object Classes

ImageNet's 1,000 classes span a wide range of real-world objects. Here is a sample organized by category:

Category	Example Classes
Animals (dogs)	Labrador, Poodle, Bulldog, Beagle, Chihuahua (120 dog breeds)
Animals (cats)	Tabby, Persian, Siamese, Egyptian cat
Animals (wild)	Lion, Tiger, Elephant, Zebra, Panda, Giraffe
Birds	Eagle, Robin, Parrot, Flamingo, Peacock, Ostrich
Reptiles	Iguana, Crocodile, Box turtle, Cobra
Food & Drink	Banana, Pizza, Hamburger, Coffee mug, Wine bottle
Vehicles	Sports car, Ambulance, Bicycle, Airplane, Canoe
Electronics	Laptop, Remote control, Cellular phone, Computer keyboard
Household	Chair, Dining table, Lamp, Toilet, Bookcase
Tools	Hammer, Screwdriver, Scissors, Measuring cup
Clothing	Bow tie, Sock, Sunglasses, Lab coat
Nature	Daisy, Mushroom, Coral reef, Volcano

4.7 How the Math Works: Softmax Output

The final layer of MobileNetV2 produces 1,000 raw scores (called *logits*). These are converted to probabilities using the **Softmax** function:

$$P(y = k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_{j=1}^{1000} e^{z_j}} \tag{4.1}$$

Where z_k is the raw score for class k, and e is Euler's number. This function guarantees that all 1,000 output probabilities sum to exactly 1.0 (100%), making them interpretable as confidence percentages.

4.8 Image Preprocessing Pipeline

Before an image can be fed to MobileNetV2, it must be transformed into the exact format the model expects:

1. **Resize** to 224×224 pixels (model's required input size)
2. **Convert to RGB** (ensure 3 color channels, no alpha channel)
3. **Normalize** each pixel: divide by 255 to get values in $[0, 1]$
4. **Standardize** using ImageNet mean and standard deviation:

$$x_{norm} = \frac{x/255 - \mu}{\sigma} \quad \text{where } \mu = [0.485, 0.456, 0.406],\ \sigma = [0.229, 0.224, 0.225] \tag{4.2}$$

5. **Transpose** from HWC format (Height×Width×Channels) to CHW format (Channels×Height×Wid for ONNX
6. **Add batch dimension**: shape becomes $(1, 3, 224, 224)$

Chapter 5

The ONNX Runtime

5.1 What is ONNX?

ONNX (Open Neural Network Exchange) is an open standard file format for machine learning models. It allows models trained in any framework (PyTorch, TensorFlow, Keras, etc.) to be exported and run using a single, efficient inference engine.

5.2 Why ONNX Instead of TFLite?

The original project was designed to use TensorFlow Lite (`tflite-runtime`), which is Google's lightweight inference engine for mobile and edge devices. However:

- `tflite-runtime` does not have pre-built packages for Python 3.13
- ONNX Runtime has full support for Python 3.11, 3.12, and 3.13
- ONNX Runtime is often *faster* than TFLite on ARM CPUs due to better use of NEON SIMD instructions

5.3 ONNX Runtime on Raspberry Pi

The `onnxruntime` package can be installed directly via pip and runs efficiently on the Pi 4's ARM Cortex-A72 CPU. It automatically uses multi-threading and hardware-optimized math libraries to maximize inference speed.

```
import onnxruntime as ort
import numpy as np

# Load the model (done once at startup)
session = ort.InferenceSession("mobilenet_v2.onnx")
input_name = session.get_inputs()[0].name

# Run inference (called for each image)
output = session.run(None, {input_name: preprocessed_image})
logits = output[0][0]  # Shape: (1000,)

# Convert to probabilities
probabilities = softmax(logits)

# Get top-5 predictions
top5 = np.argsort(probabilities)[::-1][:5]
```

Listing 5.1: Running MobileNetV2 with ONNX Runtime

Chapter 6

The Flask Web Application

6.1 What is Flask?

Flask is a lightweight Python web framework. It allows you to create web servers with just a few lines of code. In Pi Vision, Flask handles:

- Serving the HTML web interface to browsers
- Receiving uploaded images via HTTP POST requests
- Passing images to the CNN for inference
- Returning JSON results back to the browser

6.2 Application Architecture

6.3 API Endpoints

URL	Method	Input	Output
`/`	GET	None	HTML web interface
`/classify`	POST	Image file (multipart)	JSON with top-5 predictions
`/capture`	POST	None (Pi Camera)	JSON + base64 image
`/health`	GET	None	JSON status check

6.4 Example JSON Response

```
{
    "success": true,
    "top_label": "Golden Retriever",
    "inference_time_ms": 287.4,
    "fun_fact": "Dogs have a sense of smell 100,000x better than humans.
       ",
    "predictions": [
        {"label": "Golden Retriever", "confidence": 94.21},
        {"label": "Labrador Retriever", "confidence": 3.18},
```

```
        {"label": "Cocker Spaniel",    "confidence": 0.87},
        {"label": "Flat-Coated Retriever", "confidence": 0.61},
        {"label": "Chesapeake Bay Retriever", "confidence": 0.43}
    ]
}
```

Listing 6.1: Example /classify API response

Chapter 7

Complete Installation Guide

7.1 On Your PC First — Flash the SD Card

1. Download Raspberry Pi Imager: https://www.raspberrypi.com/software/
2. Insert MicroSD card into your PC
3. Select OS: *Raspberry Pi OS (64-bit, Bookworm)*
4. Click gear icon, configure WiFi + SSH + username/password
5. Flash the card (takes about 5 minutes)

7.2 Boot the Pi and Connect

```
# Wait 90 seconds after plugging in power, then:
ssh pi@raspberrypi.local

# If that fails, find the IP first (Windows PowerShell):
arp -a
# Look for a new device, then:
ssh pi@192.168.1.45
```

Listing 7.1: SSH into your Raspberry Pi

7.3 Install System Dependencies

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3-pip python3-dev libjpeg-dev zlib1g-dev
```

Listing 7.2: Install required system packages

7.4 Install Python Packages

```
pip3 install flask pillow numpy onnxruntime --break-system-packages
```

Listing 7.3: Install Python libraries

7.5 Transfer Project Files to Pi

From a *new* PowerShell window on your PC:

```
scp -r "C:\Users\YourName\Downloads\pi_vision" pi@raspberrypi.local:~/
    pi_vision
```

Listing 7.4: Copy project files from PC to Pi (run on PC)

7.6 Download the AI Model

```
cd ~/pi_vision
python3 download_models.py
```

Listing 7.5: Run on the Pi inside the project folder

7.7 Run Pi Vision

```
cd ~/pi_vision
python3 app.py
```

Listing 7.6: Start the web application

Expected output:

```
Loading MobileNetV2 ONNX model...
Model loaded!
1000 classes ready
Pi Vision is running!
Open your browser at: http://raspberrypi.local:5000
```

Open a browser on your phone or PC and navigate to:

`http://raspberrypi.local:5000`

7.8 Running on Windows PC (Without Pi)

The same app runs on any Windows/Mac/Linux computer for development and testing:

```
pip install flask pillow numpy onnxruntime
python download_models.py
python app.py
# Open: http://localhost:5000
```

Listing 7.7: Run on Windows PC

Chapter 8

Project File Structure

```
pi_vision/
  app.py                    Main Flask server and CNN inference engine
  setup_model.py            Downloads the TFLite model (original version)
  download_models.py        Downloads ONNX model and ImageNet labels
  mobilenet_v2.onnx         MobileNetV2 model weights (14 MB, auto-
     downloaded)
  imagenet_labels.txt       1000 class names (auto-downloaded)
  README.md                 Setup and usage documentation
  templates/
    index.html              Mobile-friendly web interface (HTML + CSS +
       JS)
```

Listing 8.1: Complete project directory structure

File	Language	Purpose
`app.py`	Python	Flask server, preprocessing, ONNX inference
`templates/index.html`	HTML/CSS/JS	Web UI, camera access, result display
`download_models.py`	Python	One-time model download utility
`mobilenet_v2.onnx`	Binary	Pre-trained CNN weights (3.4M parameters)
`imagenet_labels.txt`	Text	1000 class names, one per line

Chapter 9

Troubleshooting

Problem	Solution
`ssh: Could not resolve hostname raspberrypi.local`	Install Bonjour (Windows): https://support.apple.com/kb/DL999 or use IP address directly
Pi not showing in `arp -a`	WiFi credentials were not saved during flashing. Re-flash the SD card and carefully fill in WiFi settings in the gear icon menu
`ModuleNotFoundError: onnxruntime`	Run: `pip3 install onnxruntime --break-system-packages`
`tflite-runtime not found`	Your Python is 3.12+. Switch to ONNX version of `app.py`
Green LED not blinking	Possible power issue; check USB-C adapter is 5V/3A
Slow inference (>1 second)	Likely undervoltage throttling; use official power adapter
Cannot open `http://raspberrypi.local:5000` from phone	Ensure phone and Pi are on same WiFi network. Try the IP address instead
`Port 5000 already in use`	Run: `pkill -f app.py` then restart
Permission denied (SSH password)	Username must be exactly `pi` (or whatever you set in Imager); password is case-sensitive

Chapter 10

Extension Ideas for Students

Once the base project is working, students can extend it in many directions:

10.1 Hardware Extensions

- **LED Feedback:** Wire colored LEDs to GPIO pins. Green flashes if confidence $> 90\%$, yellow for 50–90%, red for $< 50\%$.
- **OLED Display:** Add a 128×64 OLED screen connected via I2C to display the top prediction without needing a phone.
- **Speaker:** Use `espeak` to have the Pi *speak* the object name out loud: `os.system(f'espeak "label"')`

10.2 Software Extensions

- **YOLOv8-Nano:** Upgrade to real-time object *detection* (draws bounding boxes around multiple objects at once).
- **Custom Classifier:** Use Google Teachable Machine (https://teachablemachine.withgoogle.com) to train your own categories (e.g., your classmates' faces, different coffee cup sizes) and export to ONNX.
- **Plant Identifier:** Swap the model for one trained on the PlantNet dataset to identify plant species from leaf photos.
- **Recycling Sorter:** Train a model to classify trash into paper, plastic, metal, glass, and organic categories.

10.3 Cloud Deployment

Deploy Pi Vision to the web for free so anyone can access it:

- **Hugging Face Spaces** — best for AI demos, free, public URL
- **Render.com** — deploy from GitHub, free tier
- **Railway.app** — beginner-friendly, free tier
- **Google Colab** — run in browser, great for teaching CNN concepts

Chapter 11

Glossary

ARM Cortex-A72
: The CPU architecture used in Raspberry Pi 4. Designed for energy-efficient computing in embedded systems.

CNN
: Convolutional Neural Network. A type of deep learning model that uses sliding filter operations to efficiently process image data.

Edge AI
: Running AI inference directly on a local device (like a Pi) rather than sending data to a cloud server.

Flask
: A lightweight Python web framework for building web servers and APIs.

GPIO
: General Purpose Input/Output. The 40-pin header on the Pi used to connect external electronics.

Headless
: Running a computer with no monitor, keyboard, or mouse, controlled entirely via SSH over the network.

ImageNet
: A dataset of 1.2 million labelled images in 1,000 categories, used to train and benchmark image classification models.

Inference
: Using a trained neural network to make predictions on new data (as opposed to training, which adjusts the model weights).

Logit
: The raw, unnormalized output score produced by the final layer of a neural network before applying softmax.

MobileNetV2
: A CNN architecture by Google optimized for mobile and embedded devices using depthwise separable convolutions.

ONNX
: Open Neural Network Exchange. A file format for storing trained neural network models in a framework-independent way.

Raspberry Pi OS
: The official Linux-based operating system for the Raspberry Pi, based on Debian.

Softmax
: A mathematical function that converts a vector of raw scores into a probability distribution summing to 1.0.

SSH Secure Shell. A protocol for securely connecting to and controlling a remote computer over a network.

TFLite TensorFlow Lite. Google's lightweight inference engine for mobile devices (replaced by ONNX Runtime in this project due to Python 3.13 support).

Transfer Learning Using a model pre-trained on a large dataset (like ImageNet) as a starting point for a new task, saving enormous amounts of training time and data.

References

1. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. *CVPR 2018*. https://arxiv.org/abs/1801.04381
2. Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., & Fei-Fei, L. (2009). ImageNet: A Large-Scale Hierarchical Image Database. *CVPR 2009*.
3. Raspberry Pi Foundation. (2024). Raspberry Pi 4 Model B Datasheet. https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-datasheet.pdf
4. ONNX Runtime Documentation. (2024). https://onnxruntime.ai/docs/
5. Pallets Projects. Flask Documentation. (2024). https://flask.palletsprojects.com/